

Rappel sur les commandes Git

Question	Réponse
C'est quoi Git ?	Git est un outil qui garde l'historique de ton code et permet de revenir en arrière.
Git est-il obligatoire pour programmer ?	Non, mais indispensable en équipe et en entreprise.
Où Git stocke l'historique du projet ?	Dans le dossier caché .git .
Que fait la commande git init ?	Elle transforme un dossier normal en dépôt Git .
Quels sont les 3 états d'un fichier dans Git ?	• Non suivi (Untracked) • Modifié • Stagé (prêt à être commit)
Quelle commande montre l'état des fichiers ?	git status
À quoi sert git add ?	À préparer les fichiers pour un commit.
Quelle est la différence entre git add . et git add fichier.js ?	• git add . → tous les fichiers • git add fichier.js → un seul fichier
À quoi sert un commit ?	À sauvegarder une version du projet .
Pourquoi le message de commit est important ?	Parce qu'il explique ce qui a été fait .
Tu modifies un fichier mais tu veux annuler. Quelle commande ?	git restore fichier.js
Est-ce que git restore supprime un commit ?	Non, il annule seulement les changements non commités .
C'est quoi une branche ?	Une copie parallèle du projet pour travailler sans casser la version principale.
Quelle est la branche par défaut ?	Main
Comment créer une branche et y aller directement ?	git switch -c ma-branche
Quelle commande montre toutes les branches ?	git branch
Quelle commande est recommandée aujourd'hui pour changer de branche ?	git switch
C'est quoi git stash ?	Un tiroir temporaire pour ranger ton travail non terminé.
Quelle commande récupère le travail stashé ?	git stash pop
Est-ce que le stash crée un commit ?	Non
À quoi sert git reset ?	À revenir en arrière dans l'historique .
Quelle est la différence entre --soft et --hard ?	• --soft → garde le code • --hard → supprime tout ⚠
Pourquoi git reset --hard est dangereux ?	Parce qu'il supprime définitivement le code .
Quelle commande affiche l'historique ?	git log
Quelle version est plus lisible pour débutant ?	git log --oneline
Tu travailles, ton chef te demande de changer de branche, mais Git refuse. Que faire ?	git stash git switch autre-branche
Tu veux tester une nouvelle idée sans casser le projet. Que fais-tu ?	Créer une nouvelle branche .

Atelier 1

1. Créer un dossier et y basculer

```
mkdir atelier-git  
cd atelier-git
```

2. Transformer le dossier en dépôt Git.

```
git init  
git status
```

3. Créer un fichier et voir les états

```
touch app.js  
git status
```

4. Préparer et sauvegarder un fichier

```
git add app.js  
git commit -m "Ajout du fichier app.js"  
git status
```

5. Annuler une modification locale

```
echo "console.log('Bonjour');" >> app.js  
git status  
git restore app.js  
git status
```

6. Travailler sur une nouvelle branche sans casser la principale

```
git branch  
git switch -c feature-todo  
git branch
```

7. Changer de branche

```
git switch main  
git status
```

8. Sauvegarder temporairement un travail en cours

```
echo "console.log('Travail non terminé');" >> app.js  
git stash
```

```
git status
git stash pop
```

9. Revenir en arrière dans l'historique

```
git commit -m "Autre commit pour test"
git log --oneline
git reset --soft HEAD~1
git status
```

- `--soft` → garde le code, `--hard` → supprime tout

Rappel :

- `git status` # où j'en suis
- `git add .` # préparer
- `git commit -m ""` # sauvegarder
- `git switch -c` # nouvelle branche
- `git branch` # voir branches
- `git stash` # ranger travail
- `git stash pop` # récupérer
- `git restore` # annuler fichier
- `git reset` # revenir en arrière
- `git log --oneline` # historique lisible

Atelier 2

Objectif : maîtriser les commandes locales

1. Crée un dossier devops-hero.

```
git init
touch hello.txt
git add .
git commit -m "Premier commit"
```

2. Crée une branche :

```
git switch -c feature/text
```

3. Ajoute du texte dans hello.txt.
4. Commit et fusionne avec main.

Objectif : gérer un dépôt distant

5. Crée un compte GitLab.
6. Crée un projet vide.
7. Connecte ton repo :

```
git remote add origin https://gitlab.com/username/devops-hero.git  
git push -u origin main
```

8. Fais une modification, puis :

```
git pull  
git push
```

Objectif : Travail en équipe – Merge Requests

9. Crée une branche feature/readme.
10. Modifie le fichier README.
11. Push la branche :

```
git push -u origin feature/readme
```

12. Sur GitLab → crée une Merge Request.
13. Simule une revue de code (ajoute un commentaire).
14. Fusionne.